Bounty Runner Operational Playbook

busconductors

November 9, 2026

CONTENTS

Bounty Runner Operational Playbook 4
1. What Bounty Runner is 4
1.1 The pipeline at a glance 4
2. OPSEC and legal 6
2.1 Authorization is mandatory 6
2.2 Stage intrusiveness 7
2.3 OPSEC practice 7
3. Prerequisites and install 8
3.1 Platform 8
3.2 External tools 8
3.3 Install Bounty Runner 10
3.4 Pre-flight check 10
3.5 nmap without running everything as root 10
4. Running an engagement, step by step 11
5. Launch presets 11
5.1 Useful CLI flags 12
6. Stage-by-stage reference 12
6.1 Recon 13
6.2 Network and edge 14
6.3 Vulnerability and content 15
6.4 Web hygiene 15
6.5 Report and tracker 16

7. Reading the output 16
7.1 summary.html: the command center 16
7.2 FINDINGS.md and findings.json 17
7.3 Raw tool output 18
8. The findings tracker 18
9. The TUI 18
9.1 Runs 18
9.2 Launch 19
9.3 Monitor 19
10. Troubleshooting 20
10.1 Unresponsive hosts “fail fast” 20
10.2 Metis hangs or never finishes 20
10.3 events.jsonl bloat 20
10.4 Other common problems 21
11. Quick reference 22

Bounty Runner Operational Playbook

Recon → scan → report → track, in one command.

This playbook takes an operator who has never used Bounty Runner through a full engagement: install, scoping, launching, monitoring, reading the results and recording findings. Every flag, file name, timeout and default in it comes from the current source tree (`runner/` and `tui/`).

> **Authorized testing only.** Run Bounty Runner only against systems you own or have **explicit written permission** to test. Read Section 2 before your first run.

---

1. What Bounty Runner is

Bounty Runner is a recon and scanning orchestrator for bug bounty hunters and penetration testers. You give it a domain. It runs a fixed 20-stage pipeline that shells out to the tools you already use (subfinder, nuclei, ffuf, katana, sqlmap, nmap, whatweb, sslyze, nikto, wafw00f and others). It then normalizes their output into one deduplicated, severity-ordered report and records the findings in a local SQLite tracker.

It ships as two pieces:

Piece	What it is	How you start it
Engine	Python CLI (`bounty_run.py`, package `runner/`)	`bounty-run example.com [flags]`
TUI	Go terminal UI (`bin/bounty-tui`) that launches, monitors and controls engine runs	`bounty-run` with no arguments, or `bin/bounty-tui`

Each run writes everything to its own directory, `outputs/<target>/<timestamp>/`. The three files you read first are **`summary.html`** (the triage command center), **`FINDINGS.md`** (the full report) and **`findings.json`** (for scripting).

1.1 THE PIPELINE AT A GLANCE


LEGEND
default-on
opt-in flag
BOUNTY RUNNER
target domain
1 . RECON
01 subs
subfinder
(+ amass --amass)
02 panel
panel-style subdomains
03 live
HTTP/S probe
04 cf
Cloudflare detect
CF only
05 origin
leak subs / dig MX / crt.sh
curl --resolve verify
live hosts + origin IPs
2 . NETWORK + EDGE
06 nmap
top-1000
(--nmap-full: -p-)
07 waf
wafw00f
08 dork
Mojeek + DuckDuckGo
09 tls
sslyze
10 nikto
nikto -maxtime
3 . VULN + CONTENT
11 vuln
nuclei
edge + origin SNI
12 content
ffuf
13 api
katana
14 param
Wayback CDX + sqlmap
15 secret
HTML + JS leaks
16 metis
metis-autocheck
4 . WEB HYGIENE
17 headers
shcheck
18 cors
6-origin probe
19 tech
whatweb
20 headi
header injection
all findings
5 . REPORT + TRACK
report
dedupe + severity sort
FINDINGS.md / findings.json
tracker
SQLite tracker.db
summary.html
command center

Green solid boxes run by default. Red dashed boxes only run when you pass their opt-in flag (or pick a preset that enables them). Diagram source: `docs/pipeline.mmd`; editable scene: `docs/pipeline.excalidraw`.

The stages always run in this order. Each one reads what earlier stages wrote:

Phase	Stages	Data handed forward
1. Recon	`subs` → `panel` → `live` → `cf` → `origin`	`subdomains.txt` → `live_hosts.txt` → Cloudflare verdict → origin IPs
2. Network + edge	`nmap` → `waf` → `dork` → `tls` → `nikto`	Live hosts and origin IPs are scanned. WAF and TLS posture are recorded
3. Vuln + content	`vuln` → `content` → `api` → `param` → `secret` → `metis`	nuclei runs against the edge **and** each origin IP (SNI/Host = target)
4. Web hygiene	`headers` → `cors` → `tech` → `headi`	Per-host header, CORS, stack and header-injection checks
5. Report + track	report → tracker → `summary.html`	Dedupe by (template, host, URL). Ingest into `outputs/tracker.db`

Two data paths matter most:

- **Live hosts drive almost everything.** `live` writes `live_hosts.txt`. `waf`, `tls`, `nikto`, `api`, `secret`, `headers`, `cors`, `tech` and `headi` all read that list. If `live` finds nothing, they fall back to the bare target.
- **Origin IPs bypass Cloudflare.** If `cf` detects Cloudflare, `origin` hunts for the real server IP. Verified origin IPs are passed to `nmap` and to an extra **nuclei pass per origin IP**. That pass connects straight to the origin while presenting the target's SNI and `Host` header, so templates hit the real application instead of the Cloudflare edge.

---

2. OPSEC and legal

2.1 AUTHORIZATION IS MANDATORY

Bounty Runner is an **active** tool. Several stages send attack traffic: `nmap` SYN/version/script scans, nuclei templates, `sqlmap` injection tests, `nikto`, header injection with `headi`, and `ffuf` brute-forcing. Running it against a system without permission can break computer-misuse law (for example the US CFAA or the UK Computer Misuse Act), get you banned from a bounty program, and disrupt production services.

Before **every** engagement:

1. **Get written authorization.** This means a bug bounty program's published scope, or a signed rules-of-engagement / statement of work for a pentest. Verbal permission is not enough.

2. **Confirm the scope.** Check that the domain, and the subdomains that `subs` will discover, are in scope. Bounty Runner scans every subdomain it finds under the target (up to `--max-hosts`), so an in-scope apex with out-of-scope subdomains needs manual handling (see 2.3).
3. **Read the program's testing rules.** Many programs ban automated scanners, DoS-adjacent load, SQL injection tooling or port scanning. Turn the matching stages off with `--skip`.
4. **Check rate-limit requirements.** If the program caps requests per second, lower concurrency (`--threads`, `--nuclei-c`) or skip the heavy stages.
5. **Keep a record.** Note the authorization reference, the date and time window, and your source IP. The run directory (with `run.json`) gives you a timestamped log of what was sent.

2.2 STAGE INTRUSIVENESS

Level	Stages	Notes
Passive	`subs`, `panel`, `dork`, `origin_deep` (crt.sh)	Third-party data sources. No packets to the target beyond DNS
Light	`live`, `cf`, `origin`, `waf`, `secret`, `headers`, `cors`, `tech`	Normal-looking HTTP requests. Low volume
Active	`vuln` (nuclei), `content` (ffuf), `api` (katana), `tls` (sslyze)	Many requests. Visible in logs and WAF dashboards
Intrusive	`nmap`, `param` (sqlmap), `nikto`, `headi`, `metis`	Attack payloads or port scans. Highest legal and stability risk

2.3 OPSEC PRACTICE

- **Source IP.** Run from a host whose IP you can name in a report. Some programs require a specific header or a registered VPN egress. Bounty Runner does not add custom headers, so use a program-approved egress if one is required.
- **Scope pruning.** To exclude out-of-scope subdomains, run the recon stages first (`--skip vuln,content,api,param,...`), review `subdomains.txt`, and then scan the in-scope hosts individually.
- **SSRF guard.** Bounty Runner only probes **publicly routable** IPs (`ipaddress.is_global`). Hosts that resolve to private, loopback or link-local addresses are dropped, so a DNS record pointing at `10.0.0.5` will not turn your scanner into an internal-network probe.
- **Target validation.** The target must be a bare domain (`example.com`). URLs are reduced to their hostname, and anything that is not a valid hostname is refused. This blocks command injection through the target argument.
- **Output is sensitive.** `outputs/` holds discovered secrets, origin IPs and vulnerabilities. It is gitignored. Keep it out of shared drives, screenshots and public repos.
- **Report responsibly.** Verify every finding by hand before you submit it. Scanner output is a lead, not proof.

---

3. Prerequisites and install

3.1 PLATFORM

Kali Linux or Debian/Ubuntu is the reference platform (package names below are Kali/Debian). macOS works for the engine with Homebrew equivalents.

Requirement	Version	Why
Python	**3.10+**	The engine uses `list[str] | None` style annotations
`requests`	any recent	The only third-party Python dependency
Go	**1.21+** (toolchain pinned in `tui/go.mod`, fetched automatically by `GOTOOLCHAIN=auto`)	Builds the TUI and the Go-based recon tools
Raw-socket rights	for `nmap` only	`nmap -sS` (SYN scan) needs raw sockets; see 3.5

3.2 EXTERNAL TOOLS

A missing tool **never crashes a run**. The stage that needs it logs `<tool> not found — skipping` and the pipeline continues. Install what your engagements need:

Tool	Stage	Default?	Install (Kali/Debian)
subfinder	`subs`	on	`go install github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest`
amass	`subs` add-on	opt-in `--amass`	see the OWASP amass repo; needs API keys to be useful
dig	`origin` (MX)	on	`sudo apt install dnsutils`
curl	`origin`, `origin_deep`, `param`	on	`sudo apt install curl`
nmap	`nmap`	opt-in `--nmap`	`sudo apt install nmap`
wafw00f	`waf`	on	`sudo apt install wafw00f` (elsewhere `pipx install wafw00f`)
sslyze	`tls`	opt-in `--tls`	`sudo apt install sslyze` (elsewhere `pipx install sslyze`)
nikto	`nikto`	opt-in `--nikto`	`sudo apt install nikto`
nuclei	`vuln`	on	`go install github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest`
ffuf	`content`	on	`go install github.com/ffuf/ffuf/v2@latest`
katana	`api`	on	`go install github.com/projectdiscovery/katana/cmd/katana@latest`
sqlmap	`param`	on (CLI)	`sudo apt install sqlmap`
metis-autocheck	`metis`	opt-in `--metis`	a shim from a local Metis install, on `PATH` (not bundled)
shcheck	`headers`	on	**vendored** (`runner/vendor/shcheck.py`), nothing to install
whatweb	`tech`	on	`sudo apt install whatweb`
headi	`headi`	opt-in `--headi`	`go install github.com/mlcsec/headi@latest`

One-shot install of the common set:

```
sudo apt install nmap whatweb nikto sqlmap dnsutils curl wafw00f sslyze
go install github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest
go install github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest
go install github.com/projectdiscovery/katana/cmd/katana@latest
go install github.com/ffuf/ffuf/v2@latest
go install github.com/mlcsec/headi@latest
echo 'export PATH="$PATH:$(go env GOPATH)/bin"' >> ~/.zshrc   # or ~/.bashrc
nuclei -update-templates
```

> On Debian 12+ and Kali, `pip install` fails with `externally-managed-environment`. Use apt or `pipx` for Python tools.

3.3 INSTALL BOUNTY RUNNER

```
git clone https://github.com/busconductors/bounty-run.git ~/tools/bounty-run
cd ~/tools/bounty-run
sudo apt install python3-requests          # or: python3 -m venv .venv &&
.venv/bin/pip install -r requirements.txt
make tui                                   # builds bin/bounty-tui
mkdir -p ~/.local/bin && ln -s "$PWD/bounty_run.py" ~/.local/bin/bounty-run
```

If you use a venv, replace the symlink with a wrapper script that runs `.venv/bin/python bounty_run.py "$@"`.

Where output goes. The CLI writes to `<repo>/outputs/`. The TUI reads `$BOUNTY_OUTPUTS`, or `~/tools/bounty-run/outputs` if that is unset. If you cloned somewhere else, set `export BOUNTY_OUTPUTS=/path/to/bounty-run/outputs` in your shell rc so CLI runs appear in the TUI.

Wordlist. `content` (ffuf) defaults to `/usr/share/wordlists/dirb/common.txt`, which exists on Kali. Elsewhere, install `dirb` or pass `--wordlist <SecLists file>`. If the wordlist is missing, the stage warns and skips.

3.4 PRE-FLIGHT CHECK

Run this before your first engagement. Every `MISSING` line is a stage that will be skipped:

```
for t in subfinder amass dig curl nmap wafw00f sslyze nikto nuclei ffuf katana \
         sqlmap metis-autocheck whatweb headi; do
  command -v "$t" >/dev/null && echo "ok      $t" || echo "MISSING $t"
done
python3 -c 'import sys, requests; assert sys.version_info >= (3, 10); print("ok
python + requests")'
ls bin/bounty-tui && bounty-run --help | head -3
```

3.5 NMAP WITHOUT RUNNING EVERYTHING AS ROOT

`nmap -sS` needs raw sockets. Do **not** run the whole pipeline with plain `sudo bounty-run`: root's `PATH` usually lacks your Go tools (so subfinder, nuclei and the rest get skipped), and the output files end up owned by root. Give nmap the capability instead and tell it to use it:

```
sudo setcap cap_net_raw,cap_net_admin,cap_net_bind_service+eip "$(command -v
nmap)"
export NMAP_PRIVILEGED=""        # add to your shell rc
```

If you must use sudo, keep your environment: `sudo env "PATH=$PATH" bounty-run example.com --nmap`.

---

4. Running an engagement, step by step

1. **Authorize and scope** (Section 2). Write down the program, the scope and any banned techniques.
2. **Pick a preset** (Section 5). Start with **Quick** on a new target, then move up to **Standard**.
3. **Launch.** Use the TUI (`bounty-run`, then press `n`) or the CLI (`bounty-run example.com --nmap --skip param`).
4. **Monitor.** Watch the stage list and live findings in the TUI Monitor. You can detach with `q` at any time; the run keeps going.
5. **Triage.** Open `outputs/<target>/<ts>/summary.html` in a browser and work the **Top findings** list top-down (Section 7).
6. **Verify** each candidate by hand: reproduce it, confirm impact, and check that it is in scope.
7. **Track.** Findings are already in `outputs/tracker.db`. Update their status as you go (Section 8).
8. **Go deeper** on interesting hosts with targeted opt-in stages (`--tls`, `--nikto`, `--headi`, `--nmap --nmap-full`) once you know the program allows them.

---

5. Launch presets

The TUI Launch screen offers four presets. They are shortcuts for stage selections, and each has an exact CLI equivalent:

Preset	Time	Stages	CLI equivalent
Quick	~5 min	Light/passive: subs, panel, live, cf, origin (+ crt.sh deep), waf, dork, secret, headers, cors, tech	`bounty-run T --skip vuln,content,api,param`
Standard *(default)*	~20–30 min	Quick **+** nmap top-1000, nuclei, ffuf, katana	`bounty-run T --nmap --skip param`
Deep	hours	Standard **+** sqlmap, sslyze, nikto, Metis, headi, amass, nmap all 65535 ports	`bounty-run T --nmap --nmap-full --tls --nikto --metis --headi --amass`
Custom	varies	Pick each stage on the next page (`space` / `x` toggles)	any mix of `--skip` and opt-in flags

> **The CLI default is not the Standard preset.** A plain `bounty-run example.com` runs every default-on stage, **including `param` (sqlmap)**, and does **not** run `nmap`. The Standard preset swaps those two: it turns `nmap` on and skips `param`. If a program bans SQL injection tooling, always pass `--skip param` on the CLI.

Times are rough. They scale with the number of live hosts, and `param`, `nikto` and `--nmap-full` dominate Deep runs.

The Launch form also takes **HTTP timeout (s)** (`--timeout`, default 8) and **Max param URLs** (`--max-param-urls`, default 50). Leave a field blank to use the engine default.

5.1 USEFUL CLI FLAGS

Flag	Default	Effect
`-o DIR`	`outputs/<target>/<ts>`	Exact output directory
`--skip a,b,c`	none	Skip stages (ids from Section 6, plus `origin_deep`)
`--timeout N`	8	Per-request HTTP probe timeout (s)
`--threads N`	30	`live` probe concurrency
`--max-hosts N`	1000	Subdomains probed by `live`; per-host stages cap at 20
`--max-param-urls N`	50	URLs sent to sqlmap
`--wordlist PATH`	dirb `common.txt`	ffuf wordlist
`--nuclei-severity`	`info,low,medium,high,critical`	nuclei severity filter
`--nuclei-c N`	25	nuclei concurrency (auto-clamped by free RAM)
`--nuclei-timeout N`	1800	nuclei wall-clock limit (s)
`--no-track`	off	Do not ingest findings into the tracker
`--interactive`	off	Prompt for output dir and engagement name
`--events PATH`	off	Write the JSON-lines event stream (the TUI sets this)

6. Stage-by-stage reference

All per-host stages respect the SSRF guard (public IPs only). Every external tool runs under a wall-clock timeout; when it expires, the tool's whole process group is killed and the stage moves on. Output files live in the run directory.

#	Stage id	Tool	Default	Output
1	`subs`	subfinder (+ amass)	on (amass opt-in)	`subfinder.txt`, `amass.txt`, `subdomains.txt`
2	`panel`	none (offline label match)	on	`panels.txt`
3	`live`	Python `requests`	on	`live_hosts.txt`
4	`cf`	Python `requests` + DNS	on	`cloudflare.json`
5	`origin`	dig, curl, crt.sh	on, **Cloudflare only**	`origin_candidates.txt`, `origin_deep.txt`
6	`nmap`	nmap	opt-in `--nmap`	`nmap.xml`, `nmap_open_ports.txt`
7	`waf`	wafw00f	on	`wafw00f_targets.txt`, `wafw00f.json`
8	`dork`	Mojeek + DuckDuckGo	on	`dork_urls.txt`, `dork.txt`
9	`tls`	sslyze	opt-in `--tls`	`sslyze/<host>.json`, `tls_scan.txt`
10	`nikto`	nikto	opt-in `--nikto`	`nikto/<host>.json`
11	`vuln`	nuclei	on	`nuclei_targets.txt`, `nuclei.jsonl`, `nuclei_origin_<ip>.jsonl`
12	`content`	ffuf	on	`content_discovery.txt`
13	`api`	katana	on	`api_endpoints.txt`
14	`param`	Wayback CDX + sqlmap	on (CLI)	`param_urls.txt`, `sqlmap_report.txt`
15	`secret`	Python `requests`	on	findings only
16	`metis`	metis-autocheck	opt-in `--metis`	`metis_targets.txt`
17	`headers`	shcheck (vendored)	on	`security_headers.json`
18	`cors`	Python `requests`	on	`cors_results.csv`
19	`tech`	whatweb	on	`whatweb.json`
20	`headi`	headi	opt-in `--headi`	`headi_raw.txt`
	report	none	always	`findings.json`, `FINDINGS.md`, `SUMMARY.md`, `summary.html`
	tracker	SQLite	on (`--no-track` disables)	`outputs/tracker.db`

6.1 RECON

1 · `subs` (subdomain enumeration). Runs `subfinder -d <target> -silent` (300 s limit). With `--amass` it also runs `amass enum -passive` (300 s). Only names equal to or under the target are kept, and the apex is always included. Writes the deduplicated list to `subdomains.txt`.

2 · `panel` (panel detection). Sends no traffic. It matches subdomain labels against admin-surface patterns: `cpanel`, `whm`, `phpmyadmin`/`pma` and `webdisk` are reported as **low**; `webmail`, `cpcalendars` and `cpcontacts` as **info**. Every panel hit is promoted to the top of the triage list in `summary.html`, whatever its severity.

3 · `live` (live host probe). For each subdomain (up to `--max-hosts`), it resolves DNS with a hard 5 s bound and drops non-public IPs. It then tries `https://` and falls back to `http://`, with redirects off and a `--timeout` (8 s) connect/read limit, using 30 threads. It records status, `Server` header and a Cloudflare flag in `live_hosts.txt`.

4 · `cf` (Cloudflare detection). Requests `https://<target>` and checks for `cf-ray`, `Server: cloudflare` and `cf-cache-status`. Writes the verdict and evidence to `cloudflare.json`.

5 · `origin` (origin-IP hunt behind Cloudflare). Runs only when `cf` says yes; otherwise it shows as skipped (`no cloudflare`). It works in four steps:

- Downloads Cloudflare's published ranges (`ips-v4`/`ips-v6`). If they cannot be loaded, the hunt **aborts** rather than produce false positives.
- Resolves 23 classic leak subdomains (`mail`, `direct`, `ftp`, `cpanel`, `mx`, `vpn`, `api`, …) and the domain's MX records (`dig +short MX`).
- Keeps public IPs **outside** Cloudflare's ranges and verifies each one with `curl --resolve <target>:443:<ip>`. The HTTP status shows whether the origin really serves the site.
- `origin_deep` (sub-step, skippable on its own) mines crt.sh certificate-transparency history for more candidates.

Verified IPs feed `nmap` and the per-origin nuclei passes.

6.2 NETWORK AND EDGE

6 · `nmap` (port scan, opt-in). Runs `nmap -T4 -sS -sV -sC --top-ports 1000 --open` against up to 32 public IPs taken from live hosts, origin IPs and the target. `--nmap-full` switches to `-p-` (all 65535 ports, which can take hours). It has a 900 s limit. `-sS` **needs raw-socket rights**: see 3.5. Each open port is an `nmap:*` finding: info by default, **low** for sensitive services (FTP, SSH, Telnet, SMB, RDP, VNC, WinRM, Docker API, MySQL/Postgres/MSSQL/Redis/Mongo/Elasticsearch/Memcached).

7 · `waf` (WAF fingerprint). Runs `wafw00f` against up to 20 live hosts (300 s). Each detected WAF is an info finding (`waf:<name>`). Knowing the WAF tells you whether nuclei/ffuf results were filtered.

8 · `dork` (search-engine dorking). Sends 4 passive queries to Mojeek and DuckDuckGo HTML, 4 s apart: admin/login URLs, exposed `sql/log/bak/conf/env` files, `index of` listings, and phpMyAdmin/cPanel/wp-admin. Only in-scope URLs are kept. `dork.txt` logs each request as `ok` or `blocked`. If every engine blocks you, the stage warns that the results are incomplete.

9 · `tls` (TLS/SSL, opt-in). Runs `sslyze` on port 443 for up to 20 hosts, 4 in parallel, with 180 s per host (about 1 min per host in practice). It checks legacy protocols, weak ciphers (RC4/DES/3DES/NULL/EXPORT/anon/MD5), Heartbleed, ROBOT, and certificate expiry (warning at 30 days), hostname match and trust. Findings are `tls:*`.

10 · `nikto` (web-server scan, opt-in). Runs `nikto` per host, up to 20 hosts and 4 in parallel. Each scan is bounded three ways: `-maxtime 300s`, a 10 s per-request timeout, and a hard kill at maxtime + 60 s. Expect about 5 minutes per host. Findings are `nikto:*`, classified by message.

6.3 VULNERABILITY AND CONTENT

11 · `vuln` (nuclei). Runs all templates at all severities against the live hosts (`nuclei_targets.txt`), and then **once per origin IP** with `-sni <target> -H "Host: <target>"`. Concurrency is `--nuclei-c` 25, clamped to available RAM at about 40 MiB per worker (floor 10) to avoid the OOM killer. Wall clock is `--nuclei-timeout` 1800 s. This is usually the longest default stage (15–30+ minutes). An interim `FINDINGS.md` is written as soon as it finishes.

12 · `content` (ffuf). Runs `ffuf -u https://<target>/FUZZ` with auto-calibration (`-ac`), 40 threads and match codes `200,204,301,302,307,401,403`, with a 600 s limit. It scans the **apex target only**. Paths go to `content_discovery.txt` and the “Discovered content” drop-down.

13 · `api` (katana). Crawls up to 20 live hosts with `katana -d 3 -jc` (JS parsing on, 300 s per host). It keeps API-looking paths (`/api/`, `/graphql`, `/v1/`, `/rest/`, …) and drops static assets. Output is `api_endpoints.txt`.

14 · `param` (Wayback + sqlmap). Pulls parameterized URLs for the domain from the Wayback CDX API, blanks their values, validates parameter names, dedupes and caps at `--max-param-urls` (50). Each URL then gets `sqlmap --batch --level 1 --risk 1 --smart` with a 300 s limit. **The worst case is 50 × 5 min**, so lower `--max-param-urls` on large targets. Confirmed injections are high-value findings, and raw output goes to `sqlmap_report.txt`.

15 · `secret` (secret / JS leak scan). Fetches each live host’s homepage plus up to 5 same-origin `<script src>` files (3 s per fetch). It looks for AWS access/secret keys, JWTs, private keys, GitHub, Slack, live Stripe and Google API keys.

16 · `metis` (curated CVE sweep, opt-in). Hands the live-host list to the external `metis-autocheck` shim, which runs check-only Metis modules against recent CVEs. It has a 600 s limit. Under the TUI it polls the control file, so **skip and cancel kill it mid-sweep**. If the shim is missing, the stage warns and skips. Findings are `metis:<module>`.

6.4 WEB HYGIENE

17 · `headers` (security headers). Runs the vendored `shcheck` (120 s) over live hosts. Missing or weak HSTS, CSP, X-Frame-Options, X-Content-Type-Options and similar headers become low/info findings. Output is `security_headers.json`.

18 · `cors` (CORS). Sends 6 probe `Origin` headers per host (5 s each): the host itself, `null`, a random foreign domain, the other scheme, and two prefix/suffix-mangled lookalikes. It flags any that are reflected or trusted, rated higher when `Access-Control-Allow-Credentials: true`. Output is `cors_results.csv`.

19 · `tech` (technology fingerprint). Runs `whatweb -a 3` (300 s). Each technology becomes an info finding and appears as a chip in the "Edge & stack" panel. Output is `whatweb.json`.

20 · `headi` (header injection, opt-in). Runs `headi` per live host, with 5 s per request and 300 s per host, sending about 169 requests per URL. Each response that changes under an injected header is a finding: `Host`, `X-Forwarded-Host` and `X-HTTP-Host-Override` hits are rated **high**, other headers **medium**. Output is `headi_raw.txt`.

6.5 REPORT AND TRACKER

After the last stage (or on cancel), the engine:

1. **Dedupes** findings by (template, host, URL) and sorts them critical → high → medium → low → info.
2. Writes `findings.json`, `FINDINGS.md` (with an executive-overview table), `SUMMARY.md` and `summary.html`.
3. Prints a `severity | title | url` table to the terminal.
4. **Ingests** new findings into `outputs/tracker.db`, skipping (target, template, URL) rows that already exist, so re-runs do not duplicate entries.

---

7. Reading the output

7.1 `SUMMARY.HTML`: THE COMMAND CENTER

Open `outputs/<target>/<ts>/summary.html` in any browser. It is self-contained (no network). Read it top-down:

Area	What it shows	What to do with it
Threat ring	Count of **critical + high** findings	Non-zero means start here today
Risk story	One sentence: candidate counts by severity, exposed panels, origin exposure, scale	Use it as the engagement headline
Metric grid	Subdomains, live hosts, critical, high, origin IPs, Cloudflare on/off	Sanity-check coverage (0 live hosts means a probe problem)
Heat bar	Proportional strip of findings by severity	A quick visual of the overall mix
Top findings	**Triage candidates**: every critical/high/medium finding **plus any** `panel:` **hit**. Panels first, then by severity. Each card ends in an action: *verify & report* (crit/high) or *review*	Your work queue. Go card by card
Attack surface	A red **"Origin IPs exposed behind Cloudflare"** alert when origins were found. *Edge & stack* (CF/WAF, API endpoints, open ports, tech chips). *Scanner coverage* (TLS issues, nikto, dork hits, panels)	Shows where to dig manually

Below these, **collapsed drop-downs** hold everything else. Click a heading to expand it:

- **Full findings (n)**: every triage candidate with URL, host and evidence.
- **Low & recon (n)**: low-severity and recon-only results (missing headers, WAF and port details).
- **Info findings (n)**: info-level nuclei, tech-detect and similar. Usually noise, but worth a scan for exposed versions.
- **Discovered content (n)**: ffuf paths.

The drop-downs are collapsed by design. The page's job is to put the real leads on screen first.

7.2 `FINDINGS.MD` AND `FINDINGS.JSON`

`FINDINGS.md` is the full, severity-ordered report. It opens with an **Executive Overview** table (scope sizes, Cloudflare and origin status, counts per severity, duration, skipped stages), followed by one block per finding with severity, title, template, host, URL and evidence. Metis findings include their module path. Paste blocks from it into bounty submissions after you verify them.

`findings.json` holds the same deduped data as a JSON array, for `jq` or your own tooling:

```
jq -r '.[] | select(.severity=="high" or .severity=="critical") | "\
(.severity)\t\(.url)"' findings.json
```

7.3 RAW TOOL OUTPUT

When a finding needs context, go to the raw files: `nuclei.jsonl` (full template match data), `sqlmap_report.txt`, `nikto/<host>.json`, `sslyze/<host>.json`, `nmap.xml` and `origin_candidates.txt`. The origin file has `IP status sources`; a `200` status with the target's content is a confirmed origin.

---

8. The findings tracker

Every run feeds `outputs/tracker.db` (SQLite) unless you pass `--no-track`. Use it to manage the whole lifecycle from a lead to a payout:

```
bounty-run track add-target example.com --program "Acme Bug Bounty"
bounty-run track list --status new                  # triage queue across all
runs
bounty-run track update 12 --status submitted       # after you file it
bounty-run track update 12 --status paid --bounty 500   # statuses: new triaged
submitted accepted paid duplicate na
bounty-run track add-finding example.com --title "IDOR on /api/v2/orders" --
severity high --url "https://..."
bounty-run track stats
bounty-run dashboard                                # cross-run
outputs/dashboard.html
```

---

9. The TUI

Start it with `bounty-run` (no arguments, on a terminal) or `bin/bounty-tui`. It has three screens.

9.1 RUNS

Lists every run under the outputs root with target, current stage, elapsed time, findings by severity and status. The header shows `N/3 active`.

Key	Action
`↑`/`k`, `↓`/`j`	Move selection
`enter`	Open the run in Monitor
`n`	New run (Launch)
`q`, `ctrl+c`	Quit

9.2 LAUNCH

A form with **Target**, **Preset**, **HTTP timeout** and **Max param URLs**. Choosing *Custom* adds a stage checklist, where `[•]` runs and `[ ]` is skipped. `enter` moves to the next field or submits, `shift+tab` goes back, `space`/`x` toggles a stage and `esc` returns to Runs.

On submit, the TUI creates the run directory and starts the engine **detached** (its own session and process group, output captured to `engine.log`). It then waits up to 3 s for the engine's `run_start` event. If the engine never starts, the TUI stops it and shows the tail of `engine.log` so you can see why (usually a missing Python dependency).

Concurrency cap: 3 active runs. A 4th launch is refused with *"3 runs already active — wait or cancel one"*. The TUI enforces this cap; the engine does not, so CLI runs are not limited.

9.3 MONITOR

Shows the stage timeline (state, duration and counts per stage), a live tool-output pane (last 5,000 lines), and a findings table that fills as findings stream in.

Key	Action
`tab`	Switch focus between output and findings
`f`	Cycle severity filter
`enter`	Finding detail (`esc` closes)
`s`	**Skip** the current stage. Asks for confirmation: `y` confirms, any other key declines
`c`	**Cancel** the run (`y` to confirm). The engine writes a **partial report** from finished stages and exits 130
`K`	**Hard kill.** Only available while the *engine not responding* banner shows
`o`	Open the report in `$PAGER` once the run has ended
`q`, `esc`	**Detach.** The run keeps going; re-attach from Runs

How control works. Skip and cancel are written as lines to the run's `control` file. The engine checks it between progress steps and while it waits on child tools, and when it acts it kills the running tool's process group. If the engine does not acknowledge within **10 s**, Monitor shows the **engine not responding** banner and enables `K`. Hard kill SIGKILLs the engine's process group and every recorded tool group (`pids`). It checks process start times first, so a reused PID is never killed. If the engine process disappears without a `run_end`, Monitor marks the run **crashed**.

Detachable runs. Quitting the TUI never stops a run. Runs started from the TUI survive the TUI exiting, an SSH disconnect (they are in their own session) and re-attaching from another terminal.

---

10. Troubleshooting

10.1 UNRESPONSIVE HOSTS “FAIL FAST”

Dead or tarpitting hosts do not stall a run. Every network step is bounded:

Layer	Bound
DNS lookup	5 s hard limit (runs in a daemon thread, because `gethostbyname` ignores socket timeouts)
HTTP probe / Cloudflare check	`--timeout` (8 s) connect + read; 10 s for `cf`
Secret / CORS fetches	3 s / 5 s per request
Each external tool	Its own wall clock (for example subfinder 300 s, nuclei 1800 s, sqlmap 300 s per URL, nikto maxtime + 60 s, sslyze 180 s per host). On expiry the tool’s **process group** gets SIGTERM, then SIGKILL after 5 s

When a stage logs `timed out after Ns`, it moves on with whatever it gathered. To speed up a target with many dead subdomains, lower `--timeout` (for example `--timeout 4`) and `--max-hosts`.

10.2 METIS HANGS OR NEVER FINISHES

- The sweep has a **600 s** hard limit. Under the TUI it also polls the control file, so `s` **(skip) or** `c` **(cancel) kills the shim and its children immediately**. Use skip rather than hard kill.
- `metis: metis-autocheck not found on PATH`: the shim is not installed or not on `PATH`. The stage skips cleanly.
- `metis: shim exited N`: Metis itself failed (usually its venv). Run `metis-autocheck file:outputs/<t>/<ts>/metis_targets.txt` by hand to see the error.
- On a plain CLI run (no `--events`), the only bound is the 600 s timeout. `Ctrl-C` stops the run but does **not** write a partial report.

10.3 `events.jsonl` BLOAT

Every line of tool output (nuclei, sqlmap, nikto, ffuf, …) is written to `events.jsonl` as an `output` event. Long Standard and Deep runs routinely reach **100–200+ MB per run**. Effects:

- **Disk usage** grows across many runs. Check it with `du -sh outputs/*/*/events.jsonl | sort -h`.
- **Slow re-attach.** Monitor replays `events.jsonl` from the start when you open a run, so very large files take longer to load.

Remedies:

- **After a run has ended**, compress or delete its `events.jsonl`. Reports (`FINDINGS.md`, `summary.html`, `findings.json`) do not depend on it. **Do not touch it while the run is active**: the TUI is tailing it.

```
find outputs -name events.jsonl -size +50M -mtime +1 -exec gzip {} \;
```

- Reduce output volume at the source: skip `param` or `nikto` unless you need them, and narrow `--nuclei-severity` (for example `low,medium,high,critical`) to drop the info flood.

10.4 OTHER COMMON PROBLEMS

Symptom	Cause	Fix
`<tool> not found — skipping`	Tool not on `PATH`	Install it; add `$(go env GOPATH)/bin` to `PATH`
`nmap` stage reports no ports / `nmap XML parse failed`	`-sS` without raw-socket rights	Grant nmap capabilities (3.5), or skip `nmap`
nuclei killed, `clamping nuclei -c` warning	Low RAM	Expected; lower `--nuclei-c` further or add swap
`content` skipped: wordlist not found	No dirb wordlist	`--wordlist /path/to/list.txt`
`dork` warns every engine blocked	Rate-limited by search engines	Results are partial; retry later from a different IP
`origin` says it could not load Cloudflare ranges	No outbound access to cloudflare.com	Fix egress; the hunt aborts on purpose to avoid false positives
0 live hosts	DNS or egress problem, or everything resolves to private IPs (SSRF guard)	Test `curl -I https://<target>`; check `live_hosts.txt`
CLI runs missing from the TUI	Different outputs root	`export BOUNTY_OUTPUTS=<repo>/outputs`
TUI: *engine did not start within 3s*	Engine crashed at startup	Read the `engine.log` tail shown; usually `requests` missing or Python < 3.10
TUI refuses to launch	3 runs active	Wait, or cancel one in Monitor

11. Quick reference

```
# Recon only, safe first look
bounty-run example.com --skip vuln,content,api,param

# Standard engagement (matches the TUI default preset)
bounty-run example.com --nmap --skip param          # see 3.5 for nmap rights

# Everything (only with explicit permission for intrusive testing)
bounty-run example.com --nmap --nmap-full --tls --nikto --metis --headi --amass

# Script against the event stream
bounty-run example.com -o outputs/example.com/manual --events
outputs/example.com/manual/events.jsonl
tail -f outputs/example.com/manual/events.jsonl | jq -c
'select(.type=="finding")'
```

Stage ids for `--skip`: `subs panel live cf origin origin_deep nmap waf dork tls nikto vuln content api param secret metis headers cors tech headi`

Opt-in flags: `--amass --nmap --nmap-full --tls --nikto --metis --headi`

Bounty Runner is MIT-licensed, © busconductors. Report vulnerabilities in Bounty Runner itself via `SECURITY.md`. Use it only on systems you are authorized to test.